

Politecnico di Torino
International University

Politecnico di Torino

Master of Science in Computer Engineering
Department of Control and Computer Engineering (DAUIN)

Large Language Models for Software Engineering

Laboratory 02: Hugging Face and Attention

T5 encoder-decoder models and attention for time-series forecasting

Course Instructor
Prof. Flavio Giobergia

Prepared by
Nicola Beccaceci

Laboratory Assistant
Claudio Savelli

Academic Year
2026/2027

Contents

1 Text-to-text generation with T5 **1**
1.1 Tokenization, special tokens, and batching 1
1.2 T5 parameters, embeddings, and encoder-decoder structure 5
1.3 Autoregressive generation 13
1.4 Cross-attention and alignment 16

2 Forecasting periodic signals with attention **21**
2.1 Synthetic data with periodic discontinuities 21
2.2 Scaled dot-product attention and attention heads 23
2.3 Encoder, forecasting model, and training 25
2.4 Inspecting attention and comparing positional representations 29

A Hugging Face implementation notes **35**

CHAPTER 1

Text-to-text generation with T5

This exercise asks how an encoder–decoder language model turns a source instruction into a target sequence, and how its tokenizer, parameters, and attention tensors expose that computation. The concrete task is to translate the prompt *translate English to German: Hello, how are you?* with the pretrained `google-t5/t5-base` checkpoint. A successful run produces a decoded translation, while inspection of token IDs, tensor shapes, model modules, and cross-attention makes each stage traceable. The accompanying lecture explains the same pipeline, including SentencePiece boundaries, padding masks, shared parameters, and decoder generation (Lab 02 transcript, 00:07:04–01:00:13).

The tokenizer maps strings to vocabulary indices and carries an attention mask that distinguishes real tokens from padding. The saved notebook cells are discussed in execution order. Their recorded outputs follow the relevant explanations when present. Outputs containing routine Hub warnings are kept as saved execution evidence, while the discussion focuses on model-relevant values. SentencePiece uses the visible marker `U+2581` for a word boundary; this is a tokenization convention, not a literal underscore character in the original text.

1.1 Tokenization, special tokens, and batching

For transformers, a fundamental step is to convert the input text into a sequence of tokens. Tokenizers are used for this purpose. Different tokenization techniques can be used (e.g., Byte-Pair Encoding). These tokenizers need to be trained on some corpus (e.g., to figure out what the most common words are). However, the Hugging Face library provides pre-trained tokenizers that can be used out of the box. Generally, each model has its own tokenizer. For example, the BertTokenizer is used for BERT models, and the GPT2Tokenizer is used for GPT-2 models. Since we will be using T5 for this exercise, we should be using the T5Tokenizer class. However, HuggingFace provides a common AutoTokenizer class that can be used to load the appropriate tokenizer for a given model (do note, however, that the returned class will be the "correct" one!).

```python
from transformers import AutoTokenizer

model_name = "google-t5/t5-base"

tokenizer = AutoTokenizer.from_pretrained(model_name)

# print the type of the tokenizer
print(type(tokenizer))
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
    rate limits and faster downloads.

<class 'transformers.models.t5.tokenization_t5.T5Tokenizer'>
```

Tokenization can be carried out by passing a string that we want to tokenize. The tokenizer implements the call method, so we can call the tokenizer directly, as follows. Note that the output is a dictionary, which generally has the following keys: - inputids: The tokenized input text (a list of token IDs by default). - attentionmask: A mask that indicates which elements in the input text are tokens and which are padding tokens. For now, we can ignore this (there is no padding). It will instead become useful when we encode batches of sentences of different lengths at the same time.

```
sentence = "hello, this is a sentence!"

# Use the tokenizer to tokenize the sentence and print the result
tokens = tokenizer(sentence)
print(tokens)
```

```

{'input_ids': [21820, 6, 48, 19, 3, 9, 7142, 55, 1], 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1]}
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.
We can reverse the encoding operation (i.e., going from token IDs to strings) by using the decode method of the tokenizer.

```
tokenizer.decode(tokens["input_ids"])
```

```

'hello, this is a sentence!</s>'
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.
To learn what the mapping between tokens and token IDs is, we can get the tokenizer's vocabulary (.getvocab()), which provides the mapping between tokens and respective IDs. For convenience, we build also a reverse vocabulary (i.e., from IDs to tokens).

```
import random

vocabulary = tokenizer.get_vocab()

# print 10 random words from the vocabulary
reverse_vocab = {v: k for k, v in vocabulary.items()}

vocab_keys = list(vocabulary.keys())
print(random.sample(vocab_keys, 10))
```

```

['<SP>editing', '<SP>compétition', '<SP>refill', '<SP>Refrigerat', '<SP>croissant', 'Lastly', '<SP>Bewohner
    ', '<SP>benefits', 'sujets', 'problem']
```

The tokenizer is loaded from the same T5 checkpoint used by the model so that token IDs and special-token conventions remain aligned throughout the exercise.

```
print("Total vocabulary size:", len(vocabulary))
```

```

Total vocabulary size: 32100
```

Let's see what the token id for the special token </s> is.

```
vocabulary["</s>"]
```

```

1
```

And indeed, note that our tokens has a 1 showing up at the end!

```
tokens["input_ids"]
```

```
[21820, 6, 48, 19, 3, 9, 7142, 55, 1]
```

We can include special tokens inside of the strings themselves. For instance:

```
tokenizer("hello!</s></s>")
```

```
{'input_ids': [21820, 55, 1, 1, 1], 'attention_mask': [1, 1, 1, 1, 1]}
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.
Instead of getting token IDs directly, we may look at the tokens being produced, directly. We use the tokenize() method in this case.

```
tokenizer.tokenize(sentence)
```

```
['<SP>hello', ',', '<SP>this', '<SP>is', '<SP>', 'a', '<SP>sentence', '!']
```

What's up with those ? They simply represent words that are starting after spaces. This helps us understand whether a token is being used at the beginning of a sentence, or if it's in the middle of a word.

```
print(tokenizer.tokenize("hello    ,world"))
print(tokenizer.tokenize("hello    , world"))
```

```
['<SP>hello', '<SP>', ',', 'world']
['<SP>hello', '<SP>', ',', '<SP>world']
```

Special tokens Each model typically has its own special tokens. Some are necessary for the training process, while others can be beneficial at inference time. Special attributes are available in the tokenizer class to access these special tokens. Some examples are: - padtoken is the token used for padding (as discussed later), - bostoken and eostoken tokens are used to indicate the beginning and end of the input text, respectively, - masktoken is used for masking tokens during training (e.g., for the masked LM task, with BERT), - septoken is used to separate sentences in the input text (e.g., next sentence prediction, with BERT), - clstoken is used to indicate the beginning of the input text (e.g., for classification tasks, with BERT), - unktoken is used to indicate unknown tokens (i.e., tokens that are not in the vocabulary). Of course,

```
tokenizer.eos_token, tokenizer.pad_token, tokenizer.bos_token
```

```
('</s>', '<pad>', None)
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.
The id suffix is used to indicate the corresponding token ID (None if not applicable).

```
tokenizer.eos_token_id, tokenizer.pad_token_id, tokenizer.bos_token_id
```

```
(1, 0, None)
```

Batch encoding/decoding In general (especially at training time) we will want to encode multiple sentences at once (e.g., an entire batch of sentences). We can pass a list of sentences to be encoded to the tokenizer.

```
sentences = [
    "this is the first sentence",
    "instead, this is the second sequence!"
]

# Use the tokenizer to tokenize the sentences and print the result
tokens = tokenizer(sentences)
print(tokens)
```

```

{'input_ids': [[48, 19, 8, 166, 7142, 1], [1446, 6, 48, 19, 8, 511, 5932, 55, 1]], 'attention_mask': [[1, 1,
     1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1]]}
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.
Of course, sentences of different lengths have a different number of tokens! However, tensors (that will be used by the model) need to have the same number of elements along each dimension. To do this, we can use padding: all sentences will be padded to the length of the longest sentence in the batch. This is done by adding pad tokens (<pad>, for T5). However, since the pad tokens are not part of the input text, we need to let the model know that it should not pay attention to them. That's what the attentionmask is for!

```
# Use the tokenizer to tokenize the sentences with padding and print the result (use padding=True)
tokens = tokenizer(sentences, padding=True)
print(tokens)
```

```

{'input_ids': [[48, 19, 8, 166, 7142, 1, 0, 0, 0], [1446, 6, 48, 19, 8, 511, 5932, 55, 1]], 'attention_mask
    ': [[1, 1, 1, 1, 1, 1, 0, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1]]}
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.
The first sentence is padded to the same length as the second sentence, with 0's (remember, the ID for <pad>!). The attention mask for the first sentence also contains 0's for the padding tokens: the model will ignore them when processing the input text. Since now all sentences have the same length, we can stack them into a single tensor. Luckily, the tokenizer can already do this for us, we just need to ask.

```
# Use the tokenizer to tokenize the sentences with padding as a tensor and print the result (use padding=
    True and return_tensors="pt")
tokens = tokenizer(sentences, padding=True, return_tensors="pt")

print(tokens["input_ids"])
print(tokens["attention_mask"])
```

```

tensor([[  48,   19,    8,  166, 7142,    1,    0,    0,    0],
        [1446,    6,   48,   19,    8,  511, 5932,   55,    1]])
tensor([[1, 1, 1, 1, 1, 1, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1, 1]])
```

Interpretation. The tensor values are the direct result of the operation above; their ordering and dimensions show how the batch or sequence is represented at this point in the computation.
For completeness, we can also decode batches of sentences, with tokenizer.batchdecode().

```
tokenizer.batch_decode(tokens["input_ids"])
```

```

['this is the first sentence</s><pad><pad><pad>',
 'instead, this is the second sequence!</s>']
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.

1.2 T5 parameters, embeddings, and encoder-decoder structure

Now we can go ahead and finally load our T5 model. We once again use a pretrained version available on HuggingFace. In general, we could use the AutoModel class for the loading, of the model. However, that version of the model does not include the specific heads for the tasks that T5 was trained on. The transformers library can make our life easier by defining a family of AutoModel... classes. For instance, the following are some commonly adopted classes: - AutoModel: the base class for all models, - AutoModelForSequenceClassification: a model for sequence classification tasks. It consists of a base model plus a classification head (linear layer + softmax). Note that, generally, the classification head is initialized randomly, and it needs to be trained on the specific task (but the library will let you know with a warning), - AutoModelFo

```
from transformers import AutoModelForSeq2SeqLM

model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

# Print the type of the model
print(type(model))
```

```

Loading weights:   0%|          | 0/257 [00:00<?, ?it/s]

<class 'transformers.models.t5.modeling_t5.T5ForConditionalGeneration'>
```

To better understand the model, we can take a look at the model's configuration. The configuration contains all the hyperparameters of the model. The configuration is available as a dictionary, and we can access the values by using the attribute notation. You can find the model's configuration object in the config attribute of the model. Or, if you only need the configuration, you can directly load it using the AutoConfig class.

```
model.config
```

```

T5Config {
  "architectures": [
    "T5ForConditionalGeneration"
  ],
  "classifier_dropout": 0.0,
  "d_ff": 3072,
  "d_kv": 64,
  "d_model": 768,
  "decoder_start_token_id": 0,
  "dense_act_fn": "relu",
  "dropout_rate": 0.1,
  "dtype": "float32",
  "eos_token_id": 1,
  "feed_forward_proj": "relu",
  "initializer_factor": 1.0,
  "is_decoder": false,
  "is_encoder_decoder": true,
  "is_gated_act": false,
  "layer_norm_epsilon": 1e-06,
  "model_type": "t5",
  "n_positions": 512,
  "num_decoder_layers": 12,
  "num_heads": 12,
  "num_layers": 12,
  "output_past": true,
  "pad_token_id": 0,
  "relative_attention_max_distance": 128,
  "relative_attention_num_buckets": 32,
  "scale_decoder_outputs": true,
  "task_specific_params": {
    "summarization": {
      "early_stopping": true,
      "length_penalty": 2.0,
```

```
      "max_length": 200,
      "min_length": 30,
      "no_repeat_ngram_size": 3,
      "num_beams": 4,
      "prefix": "summarize: "
    },
    "translation_en_to_de": {
      "early_stopping": true,
      "max_length": 300,
      "num_beams": 4,
      "prefix": "translate English to German: "
    },
    "translation_en_to_fr": {
      "early_stopping": true,
      "max_length": 300,
      "num_beams": 4,
      "prefix": "translate English to French: "
    },
    "translation_en_to_ro": {
      "early_stopping": true,
      "max_length": 300,
      "num_beams": 4,
      "prefix": "translate English to Romanian: "
    }
  },
  "tie_word_embeddings": true,
  "transformers_version": "5.15.1",
  "use_cache": true,
  "vocab_size": 32128
}
```

We can now inspect the model to better understand its architecture.

```
model
```

```

T5ForConditionalGeneration(
  (shared): Embedding(32128, 768)
  (encoder): T5Stack(
    (embed_tokens): Embedding(32128, 768)
    (block): ModuleList(
      (0): T5Block(
        (layer): ModuleList(
          (0): T5LayerSelfAttention(
            (SelfAttention): T5Attention(
              (q): Linear(in_features=768, out_features=768, bias=False)
              (k): Linear(in_features=768, out_features=768, bias=False)
              (v): Linear(in_features=768, out_features=768, bias=False)
              (o): Linear(in_features=768, out_features=768, bias=False)
              (relative_attention_bias): Embedding(32, 12)
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (1): T5LayerFF(
            (DenseReluDense): T5DenseActDense(
              (wi): Linear(in_features=768, out_features=3072, bias=False)
              (wo): Linear(in_features=3072, out_features=768, bias=False)
              (dropout): Dropout(p=0.1, inplace=False)
              (act): ReLU()
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
      (1-11): 11 x T5Block(
        (layer): ModuleList(
          (0): T5LayerSelfAttention(
            (SelfAttention): T5Attention(
              (q): Linear(in_features=768, out_features=768, bias=False)
              (k): Linear(in_features=768, out_features=768, bias=False)
              (v): Linear(in_features=768, out_features=768, bias=False)
```

```
              (o): Linear(in_features=768, out_features=768, bias=False)
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (1): T5LayerFF(
            (DenseReluDense): T5DenseActDense(
              (wi): Linear(in_features=768, out_features=3072, bias=False)
              (wo): Linear(in_features=3072, out_features=768, bias=False)
              (dropout): Dropout(p=0.1, inplace=False)
              (act): ReLU()
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (final_layer_norm): T5LayerNorm()
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (decoder): T5Stack(
    (embed_tokens): Embedding(32128, 768)
    (block): ModuleList(
      (0): T5Block(
        (layer): ModuleList(
          (0): T5LayerSelfAttention(
            (SelfAttention): T5Attention(
              (q): Linear(in_features=768, out_features=768, bias=False)
              (k): Linear(in_features=768, out_features=768, bias=False)
              (v): Linear(in_features=768, out_features=768, bias=False)
              (o): Linear(in_features=768, out_features=768, bias=False)
              (relative_attention_bias): Embedding(32, 12)
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (1): T5LayerCrossAttention(
            (EncDecAttention): T5Attention(
              (q): Linear(in_features=768, out_features=768, bias=False)
              (k): Linear(in_features=768, out_features=768, bias=False)
              (v): Linear(in_features=768, out_features=768, bias=False)
              (o): Linear(in_features=768, out_features=768, bias=False)
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (2): T5LayerFF(
            (DenseReluDense): T5DenseActDense(
              (wi): Linear(in_features=768, out_features=3072, bias=False)
              (wo): Linear(in_features=3072, out_features=768, bias=False)
              (dropout): Dropout(p=0.1, inplace=False)
              (act): ReLU()
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
      (1-11): 11 x T5Block(
        (layer): ModuleList(
          (0): T5LayerSelfAttention(
            (SelfAttention): T5Attention(
              (q): Linear(in_features=768, out_features=768, bias=False)
              (k): Linear(in_features=768, out_features=768, bias=False)
              (v): Linear(in_features=768, out_features=768, bias=False)
              (o): Linear(in_features=768, out_features=768, bias=False)
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (1): T5LayerCrossAttention(
            (EncDecAttention): T5Attention(
```

```
              (q): Linear(in_features=768, out_features=768, bias=False)
              (k): Linear(in_features=768, out_features=768, bias=False)
              (v): Linear(in_features=768, out_features=768, bias=False)
              (o): Linear(in_features=768, out_features=768, bias=False)
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (2): T5LayerFF(
            (DenseReluDense): T5DenseActDense(
              (wi): Linear(in_features=768, out_features=3072, bias=False)
              (wo): Linear(in_features=3072, out_features=768, bias=False)
              (dropout): Dropout(p=0.1, inplace=False)
              (act): ReLU()
            )
            (layer_norm): T5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (final_layer_norm): T5LayerNorm()
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (lm_head): Linear(in_features=768, out_features=32128, bias=False)
)
```

Interpretation. This module representation exposes the parameterized layers inspected by the exercise and confirms the implementation structure used by subsequent calls.
Tokens embedding

```
model.shared
```

```

Embedding(32128, 768)
```

Interpretation. This module representation exposes the parameterized layers inspected by the exercise and confirms the implementation structure used by subsequent calls.
First, we can verify that this shared embedding layer, as the name suggests, is shared between the encoder and the decoder. We do so by checking the id of the embedding layer with the embedding layers found in the encoder and decoder (model.encoder.embedtokens and model.decoder.embedtokens).

```
id(model.shared) == id(model.encoder.embed_tokens) and id(model.shared) == id(model.decoder.embed_tokens)
```

```

False
```

Next, we note that the embedding layer has 32128 tokens. This is not exactly the number of tokens we saw before (32100). The extra 28 tokens are "leftovers". 32128 = 251 128 is a more "GPU friendly" number, and it's used to speed up the computation (same reason why we often see batch sizes that are powers of 2).

```
words = [
    "chair",
    "table",
    "plate",
    "knife",
    "spoon",
    "horse",
    "goat",
    "sheep",
    "cat",
    "dog",
]
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.

This lookup retrieves the vocabulary ID assigned to T5's end-of-sequence marker, which the subsequent encoding and decoding steps use to mark sequence boundaries.

```
word_tokens = tokenizer(words, return_tensors="pt", padding=True)["input_ids"][:, 0]
print(word_tokens)
token_embeddings = model.shared(word_tokens).cpu().detach().numpy()
print(token_embeddings.shape)
```

```

tensor([ 3533,   953,  3829, 10821, 14987,  4952, 18174, 15184,  1712,  1782])
(10, 768)
```

Interpretation. The tensor values are the direct result of the operation above; their ordering and dimensions show how the batch or sequence is represented at this point in the computation. Principal-component analysis projects the selected token embeddings into two dimensions for inspection. The resulting coordinates show relative structure in this projection; they do not preserve every distance in the original embedding space.

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2)
X_pca = pca.fit_transform(token_embeddings)

fig, ax = plt.subplots(figsize=(5,5))
ax.scatter(X_pca[:, 0], X_pca[:, 1])
for i, word in enumerate(words):
    ax.text(X_pca[i, 0]+5, X_pca[i, 1]+5, word)
```

```

<Figure size 500x500 with 1 Axes>
```

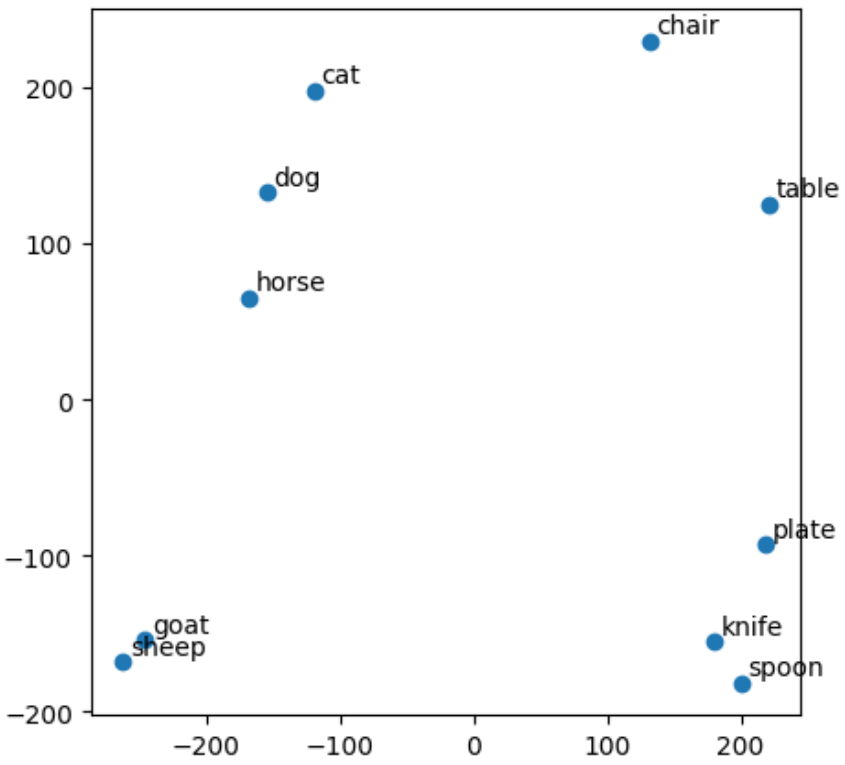


Figure 1.1: Saved notebook output.

The cosine-similarity matrix compares the directions of the selected token vectors pairwise.

Its color pattern makes stronger and weaker directional similarity visible without treating the values as probabilities.

```
from sklearn.metrics.pairwise import cosine_similarity

cosine_similarity(token_embeddings)
fig, ax = plt.subplots(figsize=(5,5))
cax = ax.imshow(cosine_similarity(token_embeddings), cmap="coolwarm")
fig.colorbar(cax)
plt.xticks(range(len(words)), words, rotation=90)
plt.yticks(range(len(words)), words);
```

```

<Figure size 500x500 with 2 Axes>
```

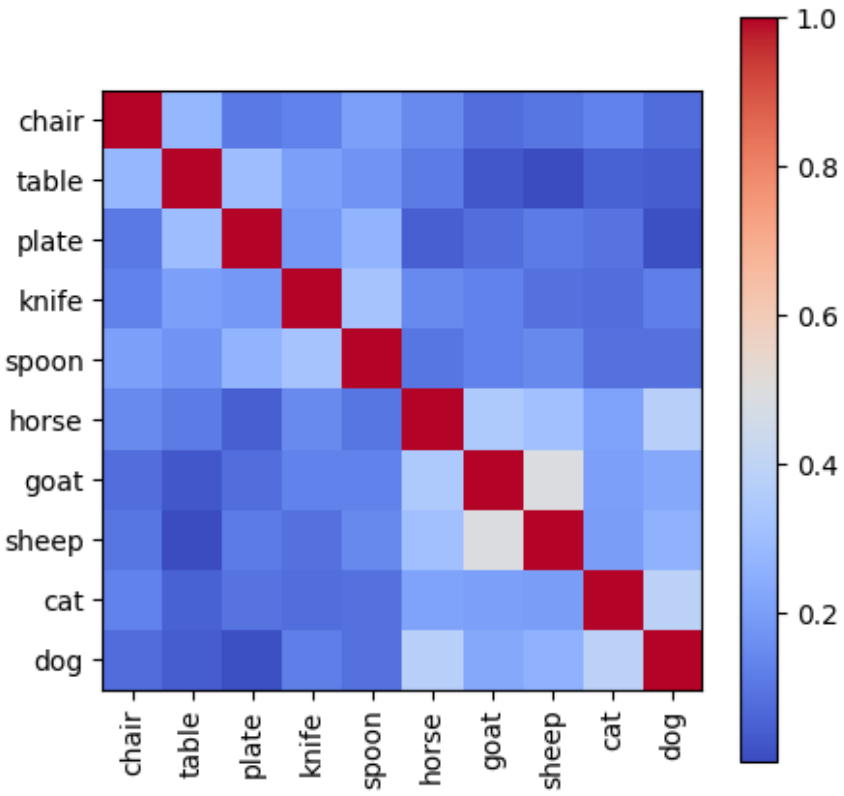


Figure 1.2: Saved notebook output.

The model's encoder contains various transformer layers (each one attention, feed-forward, and normalization layers).

```
model.encoder
```

```

T5Stack(
  (embed_tokens): Embedding(32128, 768)
  (block): ModuleList(
    (0): T5Block(
      (layer): ModuleList(
        (0): T5LayerSelfAttention(
          (SelfAttention): T5Attention(
            (q): Linear(in_features=768, out_features=768, bias=False)
            (k): Linear(in_features=768, out_features=768, bias=False)
            (v): Linear(in_features=768, out_features=768, bias=False)
            (o): Linear(in_features=768, out_features=768, bias=False)
            (relative_attention_bias): Embedding(32, 12)
          )
```

```
          (layer_norm): T5LayerNorm()
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (1): T5LayerFF(
          (DenseReluDense): T5DenseActDense(
            (wi): Linear(in_features=768, out_features=3072, bias=False)
            (wo): Linear(in_features=3072, out_features=768, bias=False)
            (dropout): Dropout(p=0.1, inplace=False)
            (act): ReLU()
          )
          (layer_norm): T5LayerNorm()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (1-11): 11 x T5Block(
      (layer): ModuleList(
        (0): T5LayerSelfAttention(
          (SelfAttention): T5Attention(
            (q): Linear(in_features=768, out_features=768, bias=False)
            (k): Linear(in_features=768, out_features=768, bias=False)
            (v): Linear(in_features=768, out_features=768, bias=False)
            (o): Linear(in_features=768, out_features=768, bias=False)
          )
          (layer_norm): T5LayerNorm()
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (1): T5LayerFF(
          (DenseReluDense): T5DenseActDense(
            (wi): Linear(in_features=768, out_features=3072, bias=False)
            (wo): Linear(in_features=3072, out_features=768, bias=False)
            (dropout): Dropout(p=0.1, inplace=False)
            (act): ReLU()
          )
          (layer_norm): T5LayerNorm()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
  (final_layer_norm): T5LayerNorm()
  (dropout): Dropout(p=0.1, inplace=False)
)
```

Interpretation. This module representation exposes the parameterized layers inspected by the exercise and confirms the implementation structure used by subsequent calls.
The block attribute is a list of T5Block modules, and contains the actual transformer layers. Indeed, we can check (as previously stated) that we have 12 encoder layers in the encoder.

```
len(model.encoder.block)
```

```

12
```

Each of the modules inside of the block is composed of a T5LayerSelfAttention module (attention layer), followed by a T5LayerFF module (feed-forward layer). Remember that there are some other details (e.g., normalization layers, or dropouts) that are shown below, but we will not discuss them in detail here. As a reminder, this is the architecture of a single encoder block.

```
model.encoder.block[0]
```

```

T5Block(
  (layer): ModuleList(
    (0): T5LayerSelfAttention(
      (SelfAttention): T5Attention(
        (q): Linear(in_features=768, out_features=768, bias=False)
        (k): Linear(in_features=768, out_features=768, bias=False)
        (v): Linear(in_features=768, out_features=768, bias=False)
```

```
        (o): Linear(in_features=768, out_features=768, bias=False)
        (relative_attention_bias): Embedding(32, 12)
      )
      (layer_norm): T5LayerNorm()
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (1): T5LayerFF(
      (DenseReluDense): T5DenseActDense(
        (wi): Linear(in_features=768, out_features=3072, bias=False)
        (wo): Linear(in_features=3072, out_features=768, bias=False)
        (dropout): Dropout(p=0.1, inplace=False)
        (act): ReLU()
      )
      (layer_norm): T5LayerNorm()
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
)
```

Interpretation. This module representation exposes the parameterized layers inspected by the exercise and confirms the implementation structure used by subsequent calls.
Decoder We can inspect the decoder in the same way. Remember the decoder's architecture!

```
model.decoder.block[0]
```

```

T5Block(
  (layer): ModuleList(
    (0): T5LayerSelfAttention(
      (SelfAttention): T5Attention(
        (q): Linear(in_features=768, out_features=768, bias=False)
        (k): Linear(in_features=768, out_features=768, bias=False)
        (v): Linear(in_features=768, out_features=768, bias=False)
        (o): Linear(in_features=768, out_features=768, bias=False)
        (relative_attention_bias): Embedding(32, 12)
      )
      (layer_norm): T5LayerNorm()
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (1): T5LayerCrossAttention(
      (EncDecAttention): T5Attention(
        (q): Linear(in_features=768, out_features=768, bias=False)
        (k): Linear(in_features=768, out_features=768, bias=False)
        (v): Linear(in_features=768, out_features=768, bias=False)
        (o): Linear(in_features=768, out_features=768, bias=False)
      )
      (layer_norm): T5LayerNorm()
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (2): T5LayerFF(
      (DenseReluDense): T5DenseActDense(
        (wi): Linear(in_features=768, out_features=3072, bias=False)
        (wo): Linear(in_features=3072, out_features=768, bias=False)
        (dropout): Dropout(p=0.1, inplace=False)
        (act): ReLU()
      )
      (layer_norm): T5LayerNorm()
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
)
```

Interpretation. This module representation exposes the parameterized layers inspected by the exercise and confirms the implementation structure used by subsequent calls.
Finally, note that the model's lmhead is a linear layer that takes the output of the decoder (768-dimensional) and maps it to the vocabulary size (32128).

```
model.lm_head
```

```
Linear(in_features=768, out_features=32128, bias=False)
```

Interpretation. This module representation exposes the parameterized layers inspected by the exercise and confirms the implementation structure used by subsequent calls.

1.3 Autoregressive generation

Token generation Let's now focus on the token generation process. Remember, we are working with an encoder-decoder architecture. The encoder processes the input text, and the decoder generates the output text. The input for the encoder is the tokenized input text. We also need to specify an input for the decoder. The decoder's input will be the currently generated sequence thus far. For the first iteration, there is nothing already generated, so we need to specify a special token to indicate the beginning of the sequence (BOS). For T5, the token will be <padtoken>.

```
import torch

input_sentence = "translate english to german: hello, how are you?"

# Use the tokenizer to tokenize the input sentence as a tensor
tokens = tokenizer(input_sentence, return_tensors="pt")

decoder_input_ids = torch.tensor([[ tokenizer.pad_token_id ]])
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.
Inspecting the encoder module exposes the stack of T5 layers that converts the input token representations into contextual states consumed by the decoder.

```
print("Encoder's input")
print(tokens["input_ids"])

print()
print("Decoder's input")
print(decoder_input_ids)
```

```
Encoder's input
tensor([[13959, 22269,    12, 13692,    10, 21820,     6,   149,    33,    25,
            58,     1]])

Decoder's input
tensor([[0]])
```

Interpretation. The tensor values are the direct result of the operation above; their ordering and dimensions show how the batch or sequence is represented at this point in the computation.
To generate the first output token (after <padtoken>), we call the model's call method (or forward method) with the input text and the decoder's input.

```
# **tokens passes the dictionary as keyword arguments (input_ids=..., attention_mask=...)
output = model(**tokens, decoder_input_ids=decoder_input_ids)
print(output.keys())
```

```
odict_keys(['logits', 'past_key_values', 'encoder_last_hidden_state'])
```

We get three outputs: - logits: The logits for each token in the vocabulary. The logits are the output of the linear layer (the lmhead) that maps the decoder's output to the vocabulary size. The logits are used to compute the probabilities of each token. - pastkeyvalues: The past key-values of the decoder. This is used to speed up the generation process for future tokens. Remember: the decoder is autoregressive, so we generate one token at a time. Since each token can only pay

attention to past tokens, it means that the predictions made for earlier tokens will not change. Thus, we can cache them and not re-compute them. So, if we pas pastkeyvalues to the model the next time we call it, it will be faster. - encoderlasthiddenstate: The hidden states of the last layer of the encoder. This is not used for generating the next token, but we may

```
print(output.logits.shape)
```

```

torch.Size([1, 1, 32128])
```

Interpretation. The reported tensor shape(s) give the batch, sequence, head, or feature dimensions returned by this exact model call; these dimensions determine which axis is interpreted in the following operation.
The shape of the logits is 1x1x32128. The first dimension is the batch size, the second dimension is the number of tokens in the output sequence (we are still at a single generated token), and the third dimension is the number of tokens in the vocabulary. So what's the next word going to be? We can use greedy decoding to take the token with the highest probability.

```
max_proba_token = output.logits[0,0].argmax()
print("Max probability token:", max_proba_token.item())
print("Corresponding token:", tokenizer.decode(max_proba_token))
```

```

Max probability token: 2501
Corresponding token: Hall
```

Looks like we're on the right track. We have a new token, which we can add to our decoderinputids tensor and continue with the next iteration.

```
# .view() acts like .reshape(), but assumes that the tensor is contiguous in memory (& is faster)
decoder_input_ids = torch.hstack([decoder_input_ids, max_proba_token.view(1, 1)])
print(decoder_input_ids)
```

```

tensor([[   0, 2501]])
```

Interpretation. The tensor values are the direct result of the operation above; their ordering and dimensions show how the batch or sequence is represented at this point in the computation.
The model call supplies the tokenized source as keyword arguments and the current decoder prefix explicitly. Its returned fields separate vocabulary logits from cached keys and values and encoder states needed by later decoding steps.

```
# Use the model to generate the next token
output = model(**tokens, decoder_input_ids=decoder_input_ids)
output.logits.shape
```

```

torch.Size([1, 2, 32128])
```

Interpretation. The reported tensor shape(s) give the batch, sequence, head, or feature dimensions returned by this exact model call; these dimensions determine which axis is interpreted in the following operation.
Now the output has shape 1x2x32128. We have generated two tokens! The first one, remember, will necessarily be the same as the one we produced before, since the masking mechanism prevents the model from seeing any following token. But just to be sure, let's decode both tokens.

```
# 0 -> first (and only) batch
max_proba_tokens = output.logits[0].argmax(axis=1)
print("Token ids", max_proba_tokens)
print("Mapped tokens", list(map(reverse_vocab.get, max_proba_tokens.tolist())))
print("Decoded string", tokenizer.decode(max_proba_tokens))
```

```

Token ids tensor([2501,   32])
Mapped tokens ['<SP>Hall', 'o']
Decoded string Hallo
```

Interpretation. The decoded sequence is the model's generated response for the saved prompt; the end marker indicates that autoregressive decoding has terminated.
We are onto something. We could go ahead and generate more tokens manually. However, we can implement our own generation loop.

```
# greedy deoding
tokenizer.batch_decode(model.generate(**tokens, do_sample=True))
```

```

/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/transformers/generation/utils.py:1676:
    UserWarning: Using the model-agnostic default `max_length` (=21) to control the generation length. We
    recommend setting `max_new_tokens` to control the maximum length of the generation.
  warnings.warn(

['<pad> Hallo, wie sind Sie?</s>']
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.
This step selects the highest-logit token at each decoder position, then maps the resulting IDs back to token strings and a decoded text sequence. The displayed sequence is the model's current greedy prediction, not a sampled alternative.

```
output.logits[0].argmax(axis=1)
```

```

tensor([2501,   32])
```

Interpretation. The tensor values are the direct result of the operation above; their ordering and dimensions show how the batch or sequence is represented at this point in the computation.
The loop repeats the model call while appending one greedy token to the decoder prefix, stopping at the end token or the 50-token limit. Because this version recomputes the full prefix each time, it makes the autoregressive state transition explicit but does not use the cache for speed.

```
decoder_input_ids = torch.tensor([[ tokenizer.pad_token_id ]])

max_length = 50
i = 0

while i < max_length and decoder_input_ids[0,-1] != tokenizer.eos_token_id:
    output = model(**tokens, decoder_input_ids=decoder_input_ids)
    max_proba_tokens = output.logits[0].argmax(axis=1)
    print(f"Step {i+1}: {tokenizer.decode(max_proba_tokens)}")
    decoder_input_ids = torch.hstack([decoder_input_ids, max_proba_tokens[-1].view(1, 1)])
    i += 1
```

```

Step 1: Hall
Step 2: Hallo

Step 3: Hallo,
Step 4: Hallo, wie

Step 5: Hallo, wie sind
Step 6: Hallo, wie sind Sie

Step 7: Hallo, wie sind Sie?
Step 8: Hallo, wie sind Sie?</s>
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.
Note that we are introducing a maxlength parameter. This is just in case, to prevent the model from generating an infinite sequence. Now, turns out that the model has a generate() method that does exactly what we just did.

```
# Use the model to generate a translation of the input sentence
out_tokens = model.generate(**tokens)
tokenizer.batch_decode(out_tokens)
```

```

['<pad> Hallo, wie sind Sie?</s>']
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.

1.4 Cross-attention and alignment

Cross-attention We can now run the previous code, but now let's compute the cross-attention weights. This will help us understand what the model is paying attention to in the input sequence, when generating the next token. To compute the attention weights, we need to reload the model, passing the outputattentions=True parameter. This will make the model return the attention weights for each layer.

```
# Load the model that returns also the output of the attentions (output_attentions=True)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name, output_attentions=True)
```

```

[transformers] The following generation flags are not valid and may be ignored: ['output_attentions']. Set `
    TRANSFORMERS_VERBOSITY=info` for more details.

Loading weights:   0%|          | 0/257 [00:00<?, ?it/s]
```

The library's generation routine performs the same iterative decoding through the model's configured generation strategy. Comparing its decoded sequence with the manual loop checks whether the explicit procedure follows the model interface's default behavior.

```
# re-prepare encoder tokens and decoder tokens
# input_sentence = "translate from english to german: hello how are you?"
input_sentence = "translate english to german: hello how are you?"

# Use the tokenizer to tokenize the input sentence as a tensor
tokens = tokenizer(input_sentence, return_tensors="pt")

# Create the token for the decoder input (start with the <pad> token -> tokenizer.pad_token_id)
decoder_input_ids = torch.tensor([[ tokenizer.pad_token_id ]])

# Use the model to generate the output putting the decoder_input_ids as the first token (decoder_input_ids=
    decoder_input_ids)
output = model(**tokens, decoder_input_ids=decoder_input_ids)
print(output.keys())
```

```

odict_keys(['logits', 'past_key_values', 'decoder_attentions', 'cross_attentions', '
    encoder_last_hidden_state', 'encoder_attentions'])
```

Decoding the argmax of the returned logits turns each output position into its most likely token under greedy selection. The result provides a direct comparison with the sequence produced by the library's generation helper.

```
# Pass in the tokenizer.batch_decode the output of the model to get the translation of the input sentence
tokenizer.batch_decode(output.logits.argmax(dim=-1))
```

```
['Hall']
```

In addition to the previous values, we now have decoderattentions, crossattentions, and encoderattentions. The names are self-explanatory. We will focus on the cross-attention to understand encoder-decoder interactions. Each of the decoder's layers has a corresponding cross-attention layer. So we expect to have 12 cross-attention layers. Indeed, the crossattention is a tuple, with 12 elements.

```
type(output.cross_attentions), len(output.cross_attentions)
```

```
(tuple, 12)
```

The shape of any one of these attentions is the following: 1x12x1x13. Remember: - the first 1 is the batch size (a single sentence), - 12 are the attention heads, each one producing an attention mask, - 1 is the number of tokens passed as input to the decoder (just BOS, for now), - 11 is the number of tokens in the input sequence.

```
output.cross_attentions[0].shape
```

```
torch.Size([1, 12, 1, 11])
```

Interpretation. The reported tensor shape(s) give the batch, sequence, head, or feature dimensions returned by this exact model call; these dimensions determine which axis is interpreted in the following operation.

Let's look at the first layer's attentions. We can see that for early stages the attention is mostly focused on identifying the task ("translate", "english", "german"). Interestingly, "german" gets attention from many of the heads, whereas "translate" and "english" are less attended to. We can expect this to be the case since T5 has been trained on various tasks, one of which was translation from english to other languages.

```
fig, ax = plt.subplots(figsize=(4,4))
ax.imshow(output.cross_attentions[0][0, :, 0].detach().numpy())
input_tokens = tokenizer.tokenize(input_sentence) + [ "</s>" ]
ax.set_xticks(range(len(input_tokens)), input_tokens, rotation=90);
```

```
<Figure size 400x400 with 1 Axes>
```

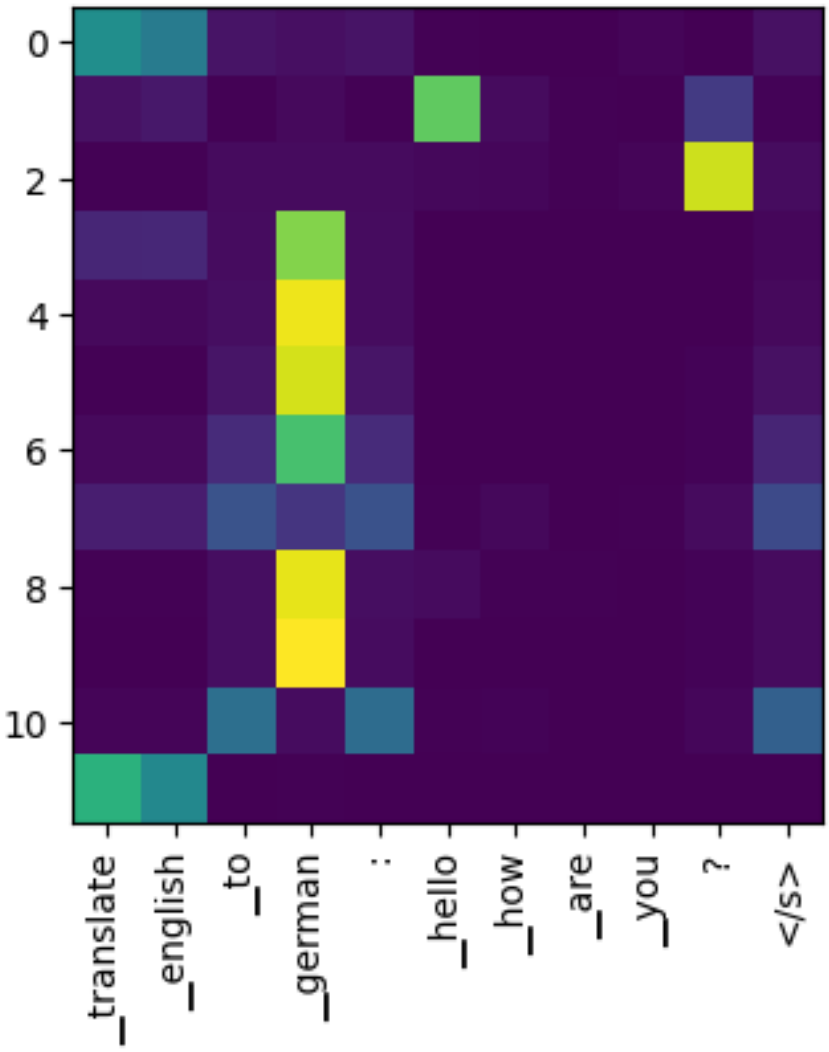


Figure 1.3: Saved notebook output.

We can check the rest of the attentions in the other layers as well. With the exception of the initial layer, the attention on "translate" and "english" is mostly gone. The attention is now focused on "german" and on the first word of the sentence ("hello").

```
fig, ax = plt.subplots(1, 10, figsize=(14, 3))

ax[0].set_ylabel("Attention head")

for i in range(10):
    ax[i].imshow(output.cross_attentions[i][0, :, 0].detach().numpy())
    input_tokens = tokenizer.tokenize(input_sentence) + [ "</s>" ]
    ax[i].set_xticks(range(len(input_tokens)), input_tokens, rotation=90);
    ax[i].set_yticks([])
```

```

<Figure size 1400x300 with 10 Axes>
```

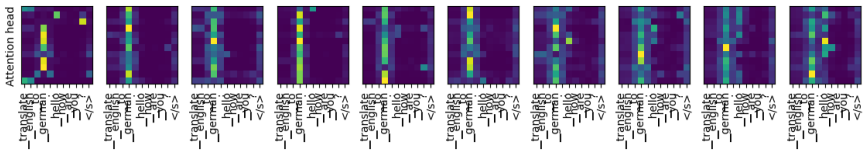


Figure 1.4: Saved notebook output.

As a final exploration, let's check the average attention across layers and heads, for the last token generated throughout the entire sequence. In this way, we can observe how the attention shifts as we generate more tokens.

```
decoder_input_ids = torch.tensor([[ tokenizer.pad_token_id ]])

attns = []

max_length = 50
i = 0

while i < max_length and decoder_input_ids[0,-1] != tokenizer.eos_token_id:
    output = model(**tokens, decoder_input_ids=decoder_input_ids)
    max_proba_tokens = output.logits[0].argmax(axis=1)
    print(f"Step {i+1}: {tokenizer.decode(max_proba_tokens)}")
    decoder_input_ids = torch.hstack([decoder_input_ids, max_proba_tokens[-1].view(1, 1)])
    attns.append(output.cross_attentions)
    i += 1
```

```

Step 1: Hall
Step 2: Hallo

Step 3: Hallo,
Step 4: Hallo, wie

Step 5: Hallo, wie sind
Step 6: Hallo, wie sind Sie

Step 7: Hallo, wie sind Sie?
Step 8: Hallo, wie sind Sie?</s>
```

Interpretation. The token IDs and mask show how the tokenizer represents the input and which positions are content, padding, or sequence boundaries.
Note: taking the average across all heads and all layers may not necessarily be the best approach (for instance, some interesting aspects may be lost in this aggregation). However, it should give us a general idea of what the model is focusing on.

```
fig, ax = plt.subplots(figsize=(5, 5))
ax.imshow(torch.stack([ torch.stack(a).mean(axis=(0, 1, 2))[-1] for a in attns ]).detach().numpy())
ax.set_xlabel("Input token")
ax.set_ylabel("Generated token")

input_tokens = tokenizer.tokenize(input_sentence) + [ "</s>" ]
output_tokens = tokenizer.tokenize(tokenizer.decode(output.logits[0].argmax(axis=1)))
ax.set_xticks(range(len(input_tokens)), input_tokens, rotation=90)
ax.set_yticks(range(len(output_tokens)), output_tokens);
```

```

<Figure size 500x500 with 1 Axes>
```

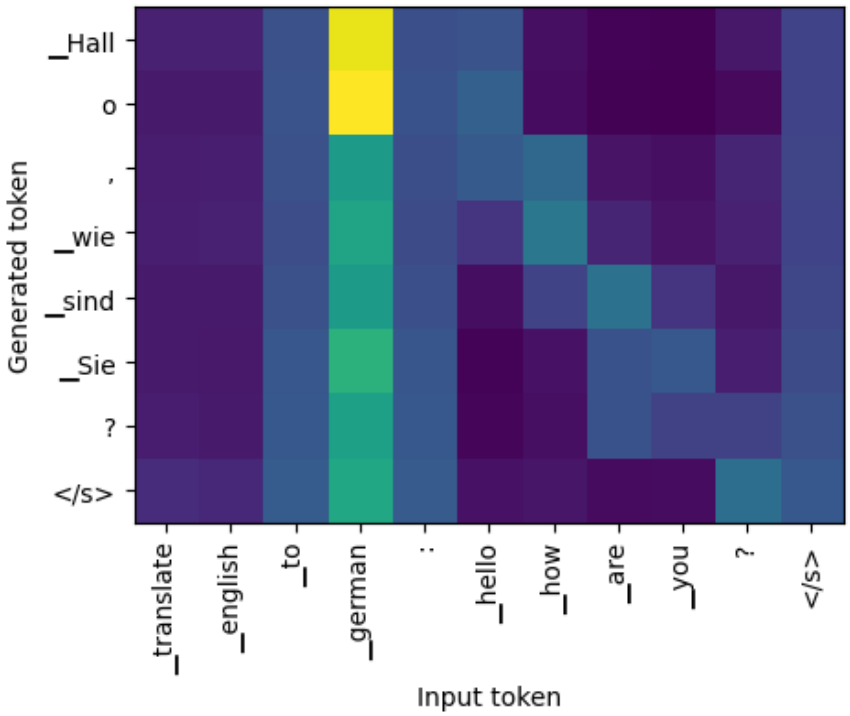


Figure 1.5: Saved notebook output.

The decoding loop makes the sequence objective explicit: the model receives an encoder input and a decoder prefix, returns vocabulary logits for the next position, and appends the selected token until the end marker or length bound. The saved outputs show the generated token sequence and the equivalent use of the library generation API. Cross-attention tensors then connect decoder positions to source tokens. The plots are qualitative summaries over selected heads or layers; averaging can conceal head-specific behavior, so they should be read as an overview rather than a causal explanation of translation.

CHAPTER 2

Forecasting periodic signals with attention

This exercise isolates attention from a pretrained language model and tests whether a small Transformer can forecast a synthetic sequence with recurring flat segments and abrupt jumps. The objective is to learn the next value from a window of observed values and compare two positional representations. The success criterion is an executed training and evaluation path with saved loss traces and attention visualizations; the positional comparison is interpreted from the recorded experiment, not assumed in advance. The lecture develops the score, normalization, and position arguments at 01:00:15–01:27:35.

2.1 Synthetic data with periodic discontinuities

The dataset generator supplies input windows and targets drawn from the same periodic jump process. Its plot documents the shape of the task before the model is introduced. The cell source below is complete, including imports and plotting statements, so the constructed signal and its visualization can be reproduced.
This inspection confirms that cross-attention is returned as a layer-indexed collection, allowing the next cells to examine alignment separately for each decoder layer.

```
# Import necessary libraries
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

import math

# set seed
torch.manual_seed(42)
np.random.seed(42)
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.
Exercise: Creating a Synthetic Time Series Dataset with Jumps In this exercise, you will implement a class to create a synthetic time series dataset. The dataset consists of flat sequences with periodic jumps, simulating data where the value remains constant for a while before increasing significantly at regular intervals. Instructions: - Implement the TimeSeriesDataset class that inherits from torch.utils.data.Dataset. - Each time series should have a predefined number of jumps, with the jumps occurring every val = 3 timesteps. - The first jump should occur at a random point between 0 and val, with the size of the jump determined by initialjump = 5 plus some random noise. - After the first jump, subsequent jumps occur every val timesteps, alternating between no jump (0) and a full jump (initialjump), again with some random noise. - The d

```
# TODO: Implement the TimeSeriesDataset class to generate a dataset of time series with jumps
class TimeSeriesDataset(torch.utils.data.Dataset):
```

```
    def __init__(self, seq_len, num_samples):
        """
        Initializes the dataset by generating a number of time series samples with jumps.

        Args:
        - seq_len (int): The length of each time series sequence.
        - num_samples (int): The number of samples in the dataset.

        Notes:
        - Each time series will have jumps every 'val' steps, starting at a random point.
        - The size of the jump is 'initial_jump' plus some random noise.
        """
        val = 3
        initial_jump = 5

        self.seq_len = seq_len
        self.data = []
        for _ in range(num_samples):
            time_series = np.zeros(seq_len + 1)

            # TODO: Introduce the first jump at a random point between 0 and val
            first_jump = np.random.randint(0, val)
            time_series[first_jump] += initial_jump + np.random.rand()  # Add a jump with some random noise

            # TODO: Continue introducing jumps every val timesteps after the first jump
            for i in range(first_jump + val, seq_len + 1, val):
                jump = 0 if i % 2 == 1 else initial_jump  # Alternate between 0 and initial_jump
                time_series[i] += jump + np.random.rand()  # Add a jump with some random noise

            self.data.append(time_series)

    def __len__(self):
        # TODO: Return the length of the dataset
        return len(self.data)

    def __getitem__(self, idx):
        """
        Returns the time series at the specified index.

        Args:
        - idx (int): The index of the desired time series.

        Returns:
        - A tuple of (time series, final jump value)
        """
        series = self.data[idx]
        return torch.tensor(series[:self.seq_len], dtype=torch.float32), torch.tensor(series[self.seq_len],
      dtype=torch.float32)
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.

The dataset class generates one synthetic sequence and its forecast target per index. A random starting phase and alternating jump pattern provide varied examples while keeping the prediction rule controlled.

```
# show a sample of the dataset

dataset = TimeSeriesDataset(seq_len=40, num_samples=1)
x, y = dataset[0]
plt.plot(range(40), x, label='x')
plt.title(f"Example Time Series Data Point\n y: {y}")
plt.scatter(40, y, color='r', label='y')
plt.legend()
plt.show()
```

```

<Figure size 640x480 with 1 Axes>
```

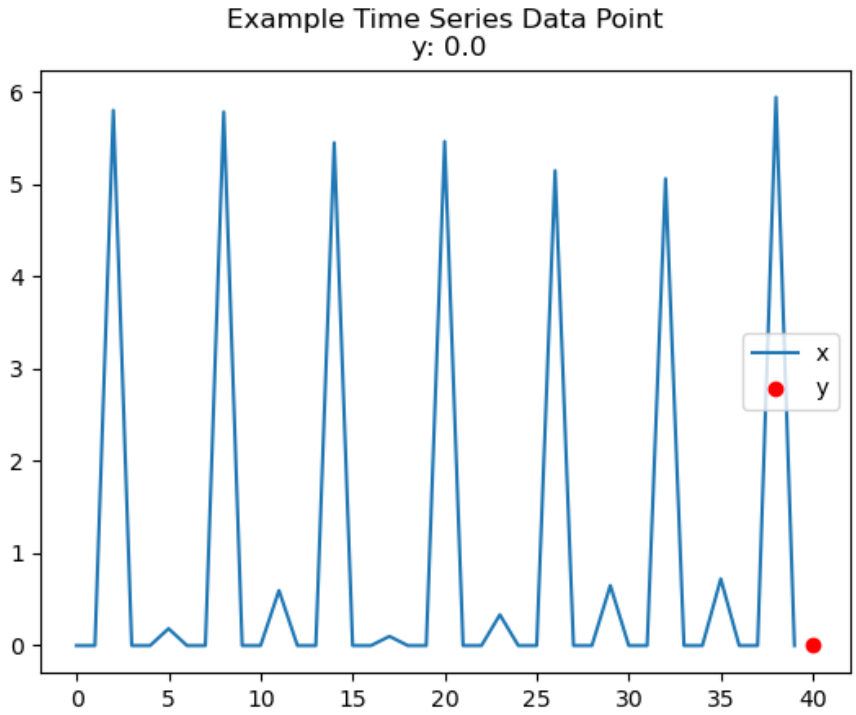


Figure 2.1: Saved notebook output.

2.2 Scaled dot-product attention and attention heads

The attention mechanism operates in the following steps: - Score Calculation: The attention scores are calculated by taking the dot product between the query and the transposed key matrices. This operation measures how relevant each position in the key is to the query. The scores are scaled by the square root of the dimensionality of the key (dk) to avoid large gradients, which could destabilize the training process. - Relative Positional Encoding: In addition to the attention scores, we add learnable relative positional encodings to capture the relationships between elements in the sequence. This encoding helps the model understand the relative positions of tokens without relying solely on absolute positional information. The relative encodings are stored in a learnable layer, which is added to the attention scores before applying softma

```
import torch
import torch.nn as nn

# Define the Scaled Dot-Product Attention Mechanism
class ScaledDotProductAttention(nn.Module):
    def __init__(self, seq_len, d_k, other_positional_encodings_present):
        super(ScaledDotProductAttention, self).__init__()

        self.d_k = d_k
        self.seq_len = seq_len
        self.other_positional_encodings_present = other_positional_encodings_present

        self.rel_layer = torch.randn(1, self.seq_len, self.seq_len, requires_grad=True) # Create a
    learnable relative positional encoding

    def forward(self, query, key, value):
        # Compute the attention scores: (Q * K^T) / sqrt(d_k)
        scores = torch.matmul(query, key.transpose(-2, -1)) / torch.sqrt(torch.tensor(self.d_k, dtype=torch.
    float32)) # Scaled Dot-Product

        # Add relative positional encodings if no other positional encodings are present: scores +
    rel_layer
```

```
        if not self.other_positional_encodings_present:
            scores = scores + self.rel_layer

        # Compute the attention weights: softmax(scores + rel_layer)
        attention_weights = torch.softmax(scores, dim=-1)

        # Compute the output: output = attention_weights * V
        output = torch.matmul(attention_weights, value)

        return output, attention_weights
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.

Exercise: Implementing an Attention Head In this exercise, you will implement the AttentionHead class, which is responsible for applying the attention mechanism to the input sequence. The attention head combines linear transformations and the Scaled Dot-Product Attention mechanism to allow the model to focus on relevant parts of the sequence. Instructions: - Implement the AttentionHead class that inherits from torch.nn.Module. - Define three linear layers for the query, key, and value matrices, which will project the input into separate subspaces. - Pass the query, key, and value matrices through the Scaled Dot-Product Attention mechanism to compute the attention output and weights. - Apply a final linear layer to the attention output to produce the final result.

```
class AttentionHead(nn.Module):
    def __init__(self, d_model, seq_len, other_positional_encodings_present):
        """
        Initializes the AttentionHead.

        Args:
        - d_model (int): The dimensionality of the input and output features.
        - seq_len (int): The length of the input sequence.
        - other_positional_encodings_present (bool): Whether other positional encodings are being used.

        Notes:
        - This class uses the Scaled Dot-Product Attention mechanism to compute attention.
        - It also applies linear transformations to the query, key, and value matrices.
        """
        super(AttentionHead, self).__init__()
        self.d_k = d_model
        self.seq_len = seq_len

        # Define linear layers for query, key, and value
        self.query_layer = nn.Linear(d_model, d_model)
        self.key_layer = nn.Linear(d_model, d_model)
        self.value_layer = nn.Linear(d_model, d_model)

        # Define the scaled dot-product attention mechanism
        self.scaled_dot_attention = ScaledDotProductAttention(self.seq_len, self.d_k,
    other_positional_encodings_present)

        # Define the final output linear layer
        self.fc_out = nn.Linear(d_model, d_model)

    def forward(self, query, key, value):
        """
        Computes the attention output and weights.

        Args:
        - query: The query matrix (shape: [batch_size, seq_len, d_model]).
        - key: The key matrix (shape: [batch_size, seq_len, d_model]).
        - value: The value matrix (shape: [batch_size, seq_len, d_model]).

        Returns:
        - output: The final output after applying attention.
        - attention_weights: The attention weights for each query/key pair.
        """

        # Apply the linear transformations to the query, key, and value matrices
        Q = self.query_layer(query)
```

```
        K = self.key_layer(key)
        V = self.value_layer(value)

        # Apply the scaled dot-product attention mechanism
        attention_output, attention_weights = self.scaled_dot_attention(Q, K, V)

        # Squeeze the attention output and pass it through the final linear layer
        output = self.fc_out(attention_output)

        return output, attention_weights
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.

2.3 Encoder, forecasting model, and training

The Transformer Encoder Block is a fundamental building block of the Transformer architecture, used to process input sequences by combining attention mechanisms and feed-forward networks. The block consists of two primary sublayers: 1. Attention Head: The input sequence is passed through the AttentionHead, which computes the scaled dot-product attention by projecting the query, key, and value matrices using linear layers. The attention output is then combined with the original input through a residual connection and normalized using LayerNorm. 2. Feed-Forward Network (FFN): After the attention sublayer, the normalized output is passed through a feed-forward network consisting of two linear layers with a ReLU activation function. The first linear layer expands the dimensionality of the input, and the second linear layer projects it back to

```
# Define a basic Transformer Encoder block
class TransformerEncoderBlock(nn.Module):
    def __init__(self, d_model, dim_feedforward, seq_len, other_positional_encodings_present, dropout=0.1):
        '''
        Initializes a basic Transformer Encoder block.

        Args:
        - d_model (int): The dimensionality of the input and output features.
        - dim_feedforward (int): The dimensionality of the feedforward network.
        - seq_len (int): The length of the input sequence.
        - other_positional_encodings_present (bool): Whether other positional encodings are being used.
        - dropout (float): The dropout probability.

        Notes:
        - This class uses the AttentionHead defined above.
        '''
        super(TransformerEncoderBlock, self).__init__()
        self.attention = AttentionHead(d_model, seq_len, other_positional_encodings_present)
        self.norm1 = nn.LayerNorm(d_model)

        # Define the second layer normalization
        self.norm2 = nn.LayerNorm(d_model)

        # Feed-forward network
        self.ffn = nn.Sequential(
            nn.Linear(d_model, dim_feedforward),
            nn.ReLU(),
            nn.Linear(dim_feedforward, d_model)
        )

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        '''
        Passes the input through the Transformer Encoder block.

        Args:
        - x: The input tensor (shape: [batch_size, seq_len, d_model]).

        Returns:
```

```
        - output: The output tensor (shape: [batch_size, seq_len, d_model]).
        - attention_weights: The attention weights for each query/key pair.
        '''

        # Attention head
        attention_output, attention_weights = self.attention(x, x, x)
        x = self.norm1(x + self.dropout(attention_output))

        # Pass the output through the feed-forward network
        ffn_output = self.ffn(x)

        # Apply the second layer normalization with the residual connection
        output = self.norm2(x + self.dropout(ffn_output))

        return output, attention_weights
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.
Finally the Transformer Model designed for time series data is implemented. Below are the key components of the model: 1. Embedding Layer: The input time series is first passed through a linear embedding layer that projects the input features into a higher-dimensional space (dmodel). This transformation enables the model to work with rich feature representations of the input data. 2. Positional Encoding: Since the Transformer does not inherently capture the order of the input sequence, positional encodings are added to provide information about the position of each time step in the sequence. If a positional encoding method is provided, it is applied to the embedded input. Otherwise, the Relative Positional Encoding provided in ScaledDotProductAttention is applied. 3. Transformer Encoder Layers: The model contains numlayers TransformerEnco

```
# Define a simple Transformer Model
class Transformer(nn.Module):
    def __init__(self, input_size, d_model, dim_feedforward, num_layers, seq_len, positional_encoding =
     None, dropout=0.1, max_len=5000):
        super(Transformer, self).__init__()

        self.positional_encoding = positional_encoding

        # Embedding for the time series input
        self.embedding = nn.Linear(input_size, d_model)

        # Positional encoding for time series data
        if self.positional_encoding is not None:
            self.pos_encoder = self.positional_encoding(d_model, max_len)

        # Transformer encoder layers
        self.encoder_layers = nn.ModuleList(
            [TransformerEncoderBlock(d_model, dim_feedforward, seq_len, False if positional_encoding is
     None else True, dropout) for _ in range(num_layers)]
        )

        # Output layer to predict a single value
        self.fc_out = nn.Linear(d_model, 1)

    def forward(self, x):
        # x shape: [batch_size, seq_len, input_size]

        # Embed the input time series data
        x = self.embedding(x)  # Shape: [batch_size, seq_len, d_model]

        # Add positional encoding
        x = self.pos_encoder(x) if self.positional_encoding is not None else x

        # List to store attention weights from all layers
        all_attention_weights = []

        # Pass through each Transformer encoder layer
        for layer in self.encoder_layers:
            x, attention_weights = layer(x)
            all_attention_weights.append(attention_weights)
```

```
        # Apply final linear layer to get the output
        output = self.fc_out(x[:, -1])  # Take the output of the last time step (sequence regression)
        return output, all_attention_weights  # Return the final output and the attention weights
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.
The module computes query–key scores, scales them by the key dimension, adds its learnable relative-position term when applicable, and uses the normalized weights to combine values. This is the core operation the later model layers reuse.

```
input_size = 1  # Number of input features
seq_len = 40 # Length of the time series sequence

d_model = 64  # Embedding dimension
dim_feedforward = 256  # Feedforward network dimension
num_layers = 3  # Number of Transformer encoder layers

model = Transformer(input_size, d_model, dim_feedforward, num_layers, seq_len)
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.
The training helper switches the model to training mode, iterates over batches for each epoch, computes the mean-squared error, and applies Adam updates. It records the epoch losses so the two positional-encoding runs can be compared using the same procedure.

```
def train_model(model, dataloader, criterion, optimizer, num_epochs=10):
    """
    Trains the given model on the provided dataset.

    Args:
    - model: The model to be trained.
    - dataloader: A DataLoader object to iterate over the dataset.
    - criterion: The loss function used to compute the training loss.
    - optimizer: The optimizer used to update the model parameters.
    - num_epochs: The number of training epochs.

    Returns:
    - loss_history: A list containing the loss value for each epoch.
    """

    # Set the model to training mode
    model.train()

    # Initialize an empty list to store the loss history
    loss_history = []

    # Loop through the epochs
    for epoch in range(num_epochs):
        total_loss = 0

        # Iterate over the batches of the dataloader
        for inputs, target in dataloader:
            # Add a feature dimension to the input data (input_size is assumed to be 1)
            inputs = inputs.unsqueeze(-1)  # Shape: [batch_size, seq_len, input_size]

            # Zero the gradients of the optimizer
            optimizer.zero_grad()

            # Perform a forward pass through the model and compute the loss
            outputs, _ = model(inputs)
            loss = criterion(outputs.squeeze(), target)

            # Perform backpropagation to compute the gradients
            loss.backward()

            # Update the model's parameters using the optimizer
            optimizer.step()

            # Accumulate the total loss for this batch
            total_loss += loss.item()
```

```
        # Compute the average loss for the current epoch
        avg_loss = total_loss / len(dataloader)

        loss_history.append(avg_loss)

        print(f'Epoch {epoch+1}/{num_epochs}, Loss: {avg_loss:.4f}')

    plt.plot(loss_history)
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Loss During Training')
    plt.show()

    return loss_history
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.
These settings fix the batch size, number of generated examples, and training duration for the experiment, so the positional-encoding comparison uses the same workload in both runs.

```
batch_size = 128 # Number of samples in each batch
num_samples = 1000 # Number of samples in the dataset
num_epochs = 15 # Number of training epochs
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.
This cell samples 1,000 synthetic sequences, batches them in groups of 128, and trains the relative-position model with mean-squared error and Adam at learning rate 0.0005. The saved losses and plot therefore show how this model fits the jump-prediction task over 15 epochs.

```
# Initialize the DataLoader
dataset = TimeSeriesDataset(seq_len=seq_len, num_samples=num_samples)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=batch_size, shuffle=True)

# Instantiate the criterion (MSE loss) and the optimizer (Adam) with a learning rate of 0.0005
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.0005)

# Train the model
train_model(model, dataloader, criterion, optimizer, num_epochs=num_epochs)
```

```

Epoch 1/15, Loss: 7.3971

Epoch 2/15, Loss: 6.1423

Epoch 3/15, Loss: 5.3159

Epoch 4/15, Loss: 4.4852

Epoch 5/15, Loss: 3.7367

Epoch 6/15, Loss: 2.3744

Epoch 7/15, Loss: 1.0627

Epoch 8/15, Loss: 0.3977

Epoch 9/15, Loss: 0.2739

Epoch 10/15, Loss: 0.1367

Epoch 11/15, Loss: 0.0736

Epoch 12/15, Loss: 0.0533

Epoch 13/15, Loss: 0.0436

Epoch 14/15, Loss: 0.0380
```

```
Epoch 15/15, Loss: 0.0363

<Figure size 640x480 with 1 Axes>

[7.397115707397461,
 6.142259895801544,
 5.315905928611755,
 4.485172241926193,
 3.7366731762886047,
 2.3743545711040497,
 1.0627223923802376,
 0.3977093156427145,
 0.273893628269434,
 0.13665180560201406,
 0.07355692842975259,
 0.05328151909634471,
 0.04360803682357073,
 0.03800722723826766,
 0.03626420395448804]
```

Interpretation. The printed loss values track optimization over 15 epochs: they move from 7.3971 to 0.0363, showing the training trajectory recorded for this run.

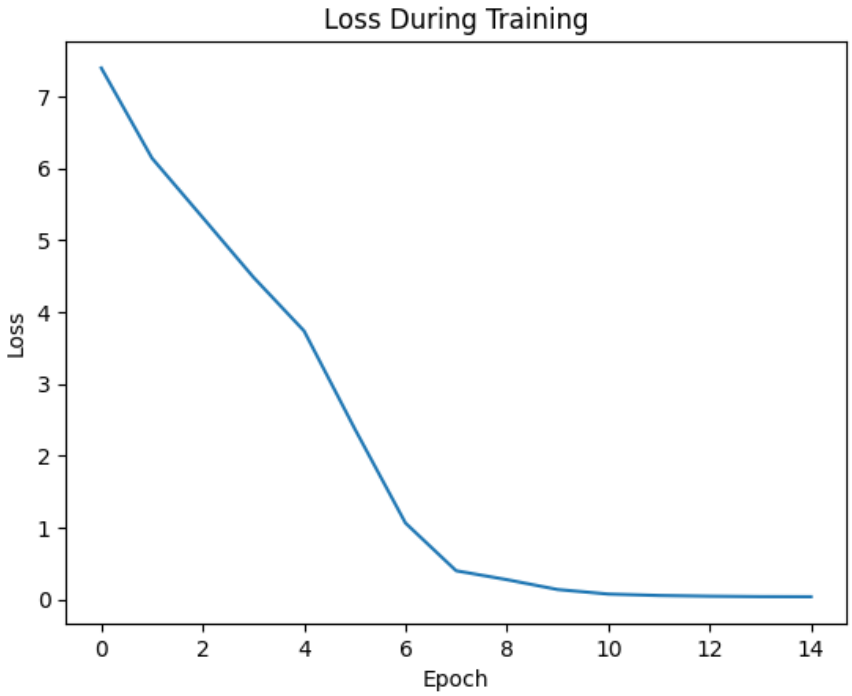


Figure 2.2: Saved relative-position model training loss.

The curve falls from about 7.4 to 0.04, with the steepest reduction during the middle epochs and a gradual flattening near the end. This is consistent with the printed epoch losses and indicates that the model fits the synthetic jump pattern in this run.

2.4 Inspecting attention and comparing positional representations

This plot displays the Input Series (blue line) along with the corresponding Normalized Attention Weights (orange dashed line). The normalized attention weights highlight the time steps that the Transformer model focuses on during the prediction process. What's interesting is how the model places more attention on previous jumps in the sequence. These small spikes in the attention weights indicate that the model is focusing on key points in the input that help it make

a more accurate prediction. The red dot marks the true target value (5.51), and the green dot shows the predicted value (5.63). The fact that the predicted value is so close to the true target demonstrates how well the model is learning from the input data.

```
import matplotlib.pyplot as plt
import numpy as np
import torch

# Visualize the Attention Weights
def visualize_attention(model, dataset):
    model.eval()
    for i in range(len(dataset)):
        inputs, target = dataset[i]
        if target != 0:
            # Reshape the input for the Transformer: [batch_size, seq_len, input_size]
            inputs = inputs.unsqueeze(0).unsqueeze(2)
            outputs, attention_weights = model(inputs)
            break

    # Plot the input time series (after undoing the batch and input dimension)
    plt.plot(inputs.permute(1, 0, 2).squeeze().numpy(), label="Input Series")

    # The attention weights are returned from all layers, so let's use the attention from the last layer
    attention_weights = attention_weights[-1][0].detach().numpy()  # [batch_size, num_heads, seq_len,
     seq_len]

    # Normalize attention weights for better visibility
    normalized_attention_weights = (attention_weights - np.min(attention_weights)) / (np.max(
     attention_weights) - np.min(attention_weights))


    # Plot the normalized attention weights
    plt.plot(normalized_attention_weights.mean(axis=0), label="Normalized Attention Weights", linestyle='--'
     )
    plt.scatter(seq_len, target, color='r', label='True Target')
    plt.scatter(seq_len, outputs[-1].item(), color='g', label='Predicted Target')
    plt.title(f"Example of a prediction:\nTrue Target: {target.item():.2f}, Predicted: {outputs[-1].item()
     :.2f}")
    plt.legend(loc='upper left')
    plt.show()



visualize_attention(model, dataset)
```

```

<Figure size 640x480 with 1 Axes>
```

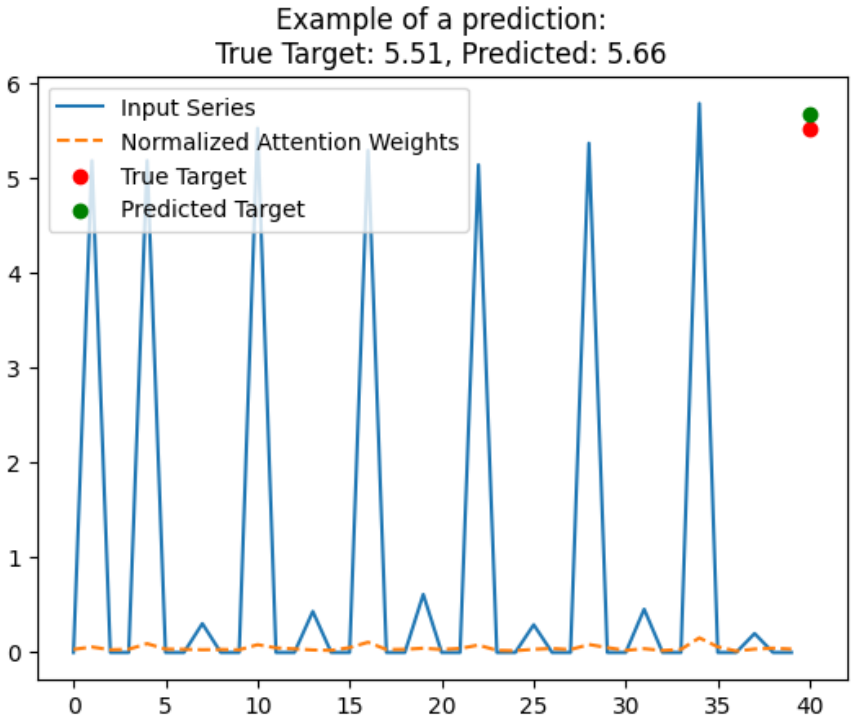


Figure 2.3: Saved notebook output.

The PositionalEncoding class shown below implements absolute positional encoding, where each position in the sequence is encoded using a combination of sine and cosine functions. This encoding is then added to the input embeddings to provide the model with information about the position of each token in the sequence. Here's how the code works: - Matrix Creation: A positional encoding matrix pe of shape (maxlen, dmodel) is initialized, where maxlen is the maximum sequence length and dmodel is the dimensionality of the model. - Position Calculation: The position tensor is created, holding the position values for each token in the sequence. The divterm tensor calculates the scaling factor for the sine and cosine functions based on the position and the model's dimensionality. - Sine and Cosine Application: The sine function is applied to the

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super(PositionalEncoding, self).__init__()

        # Create a matrix of shape (max_len, d_model) to hold the positional encodings
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)

        # Compute sine and cosine positional encodings
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)  # Apply sine to even indices in the encoding
        pe[:, 1::2] = torch.cos(position * div_term)  # Apply cosine to odd indices in the encoding

        pe = pe.unsqueeze(0).transpose(0, 1)  # Reshape for batch compatibility
        self.register_buffer('pe', pe)  # Register pe as a buffer so it doesn't get updated during
    backpropagation

    def forward(self, x):
        # Add positional encoding to the input
        return x + self.pe[:x.size(0), :].repeat(1,x.shape[1],1)
```

No standalone display output was recorded; this cell's effect is carried forward in subsequent tensors, model state, or training results.
The visualization selects a nonzero target, runs the trained model, and plots the input alongside

normalized attention weights. The true and predicted target markers make it possible to inspect one forecast together with the time steps receiving attention.

```
# TODO: Initialize the Transformer model with the PositionalEncoding class
model = Transformer(input_size, d_model, dim_feedforward, num_layers, seq_len, positional_encoding=
     PositionalEncoding)

# TODO: Instantiate the criterion (MSE loss) and the optimizer (Adam) with a learning rate of 0.0005
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.0005)

# TODO: Train the model
train_model(model, dataloader, criterion, optimizer, num_epochs=num_epochs)
```

```

Epoch 1/15, Loss: 7.1401

Epoch 2/15, Loss: 6.5387

Epoch 3/15, Loss: 6.2578

Epoch 4/15, Loss: 5.9696

Epoch 5/15, Loss: 5.8090

Epoch 6/15, Loss: 5.7272

Epoch 7/15, Loss: 5.7824

Epoch 8/15, Loss: 5.6585

Epoch 9/15, Loss: 5.5258

Epoch 10/15, Loss: 5.6687

Epoch 11/15, Loss: 5.5961

Epoch 12/15, Loss: 5.4340

Epoch 13/15, Loss: 5.3413

Epoch 14/15, Loss: 5.3534

Epoch 15/15, Loss: 5.4174

<Figure size 640x480 with 1 Axes>

[7.140081226825714,
 6.538714528083801,
 6.257763028144836,
 5.96960574388504,
 5.808981239795685,
 5.727154850959778,
 5.78239518404007,
 5.658481121063232,
 5.525805592536926,
 5.668729066848755,
 5.596091151237488,
 5.433995604515076,
 5.341306388378143,
 5.353363215923309,
 5.417354047298431]
```

Interpretation. The printed loss values track optimization over 15 epochs: they move from 7.1401 to 5.4174, showing the training trajectory recorded for this run.

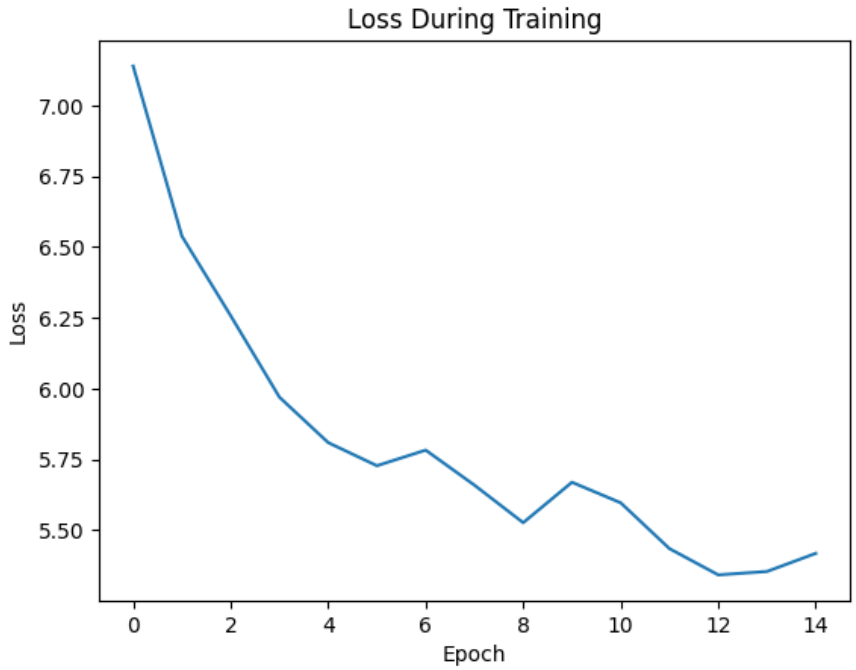


Figure 2.4: Saved absolute-sinusoidal model training loss.

This run decreases initially but levels off near 5.4, unlike the relative-position run. Under the shared synthetic-data setup and 15-epoch budget, the absolute-encoding configuration therefore learns the target less effectively; the result is specific to this implementation and experiment. This final visualization applies the same diagnostic to the second positional-encoding run, making its attention pattern and one-example prediction comparable with the relative-position model above.

```
import matplotlib.pyplot as plt
import numpy as np
import torch

# Visualize the Attention Weights
def visualize_attention(model, dataset):
    model.eval()
    for i in range(len(dataset)):
        inputs, target = dataset[i]
        if target != 0:
            # Reshape the input for the Transformer: [batch_size, seq_len, input_size]
            inputs = inputs.unsqueeze(0).unsqueeze(2)
            outputs, attention_weights = model(inputs)
            break

    # Plot the input time series (after undoing the batch and input dimension)
    plt.plot(inputs.permute(1, 0, 2).squeeze().numpy(), label="Input Series")

    # The attention weights are returned from all layers, so let's use the attention from the last layer
    attention_weights = attention_weights[-1][0].detach().numpy()  # [batch_size, num_heads, seq_len,
     seq_len]

    # Normalize attention weights for better visibility
    normalized_attention_weights = (attention_weights - np.min(attention_weights)) / (np.max(
     attention_weights) - np.min(attention_weights))


    # Plot the normalized attention weights
    plt.plot(normalized_attention_weights.mean(axis=0), label="Normalized Attention Weights", linestyle='--'
     )
    plt.scatter(seq_len, target, color='r', label='True Target')
    plt.scatter(seq_len, outputs[-1].item(), color='g', label='Predicted Target')
    plt.title(f"Example of a prediction:\nTrue Target: {target.item():.2f}, Predicted: {outputs[-1].item()
     :.2f}")
```

```
    plt.legend(loc='upper left')
    plt.show()


visualize_attention(model, dataset)
```

```

<Figure size 640x480 with 1 Axes>
```

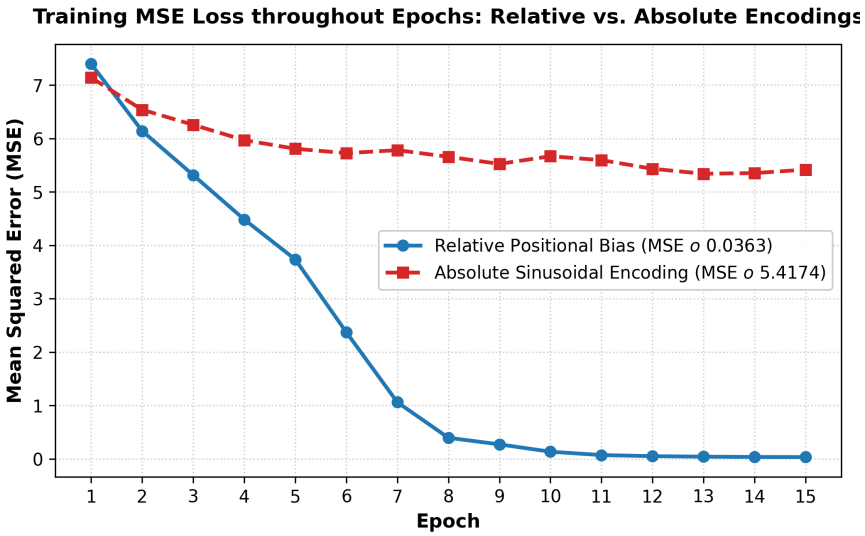


Figure 2.5: Saved notebook output.

The custom model projects each scalar observation into a hidden representation, applies attention and a feed-forward transformation, and maps the representation to a forecast. Training outputs report the saved epoch losses. The final plots compare attention allocation and the two positional schemes under the notebook's fixed settings; the relative-position run reaches a much lower recorded loss than the absolute sinusoidal run in this execution. This result is evidence for this synthetic task and configuration, not a general guarantee about positional encodings.

Appendix A

Hugging Face implementation notes

The notebook uses the
texttttransformers API to load a pretrained tokenizer and conditional generation model. The returned object exposes configuration, model modules, logits, encoder states, and optional attention values through named outputs. The complete code that invokes these interfaces is included in the T5 exercise chapters above, next to the corresponding results. In practical generation,
textttgenerate() manages iterative decoding and cache use; a manually written loop is useful here because it exposes how prefixes grow and logits are selected. The appendix is limited to interpreting interfaces actually used in this laboratory.